Лабораторна робота №4

Мета роботи: Сформувати навички інтеграції сенсорів у ROS2, реалізувати сенсорну ноду, організувати публікацію вимірювань у Topic та застосувати базову обробку (фільтрацію) даних.

Посилання на репозиторій: [Anastasia11070830/ROS2]

Хід роботи

1. Нова віртуальна машина не потрібна: працюємо в тому самому середовищі що і минулі лабораторні роботи.
2. **Створення пакету Sensor Node.**
   Спочатку в терміналі виконуються команди для активації середовища ROS 2, після чого відбувається перехід у папку, де й зберігаються вихідні коди. Далі використовується головна команда ros2 pkg create, яка автоматично генерує всі необхідні файли. У цій команді за допомогою прапорця --build-type ament_python вказується, що пакет створюється для мови Python, а --license Apache-2.0 додає відповідний файл ліцензії.

   Після створення пакету за допомогою команди tree виводиться його файлова структура для перевірки. У корені пакету знаходяться файли LICENSE та package.xml (головний маніфест із метаданими та залежностями). Також є папка resource з файлом-маркером, необхідним системі збірки ament. Основна робоча директорія також називається sensor_pkg, всередині неї лежать файли __init__.py та безпосередньо сам скрипт вузла sensor_node.py, який наразі містить лише базову заглушку. Крім того, згенеровано конфігураційні файли setup.cfg та setup.py, які відповідають за процес збірки й інсталяції, а також папку test із заготовками для стандартних перевірок коду.

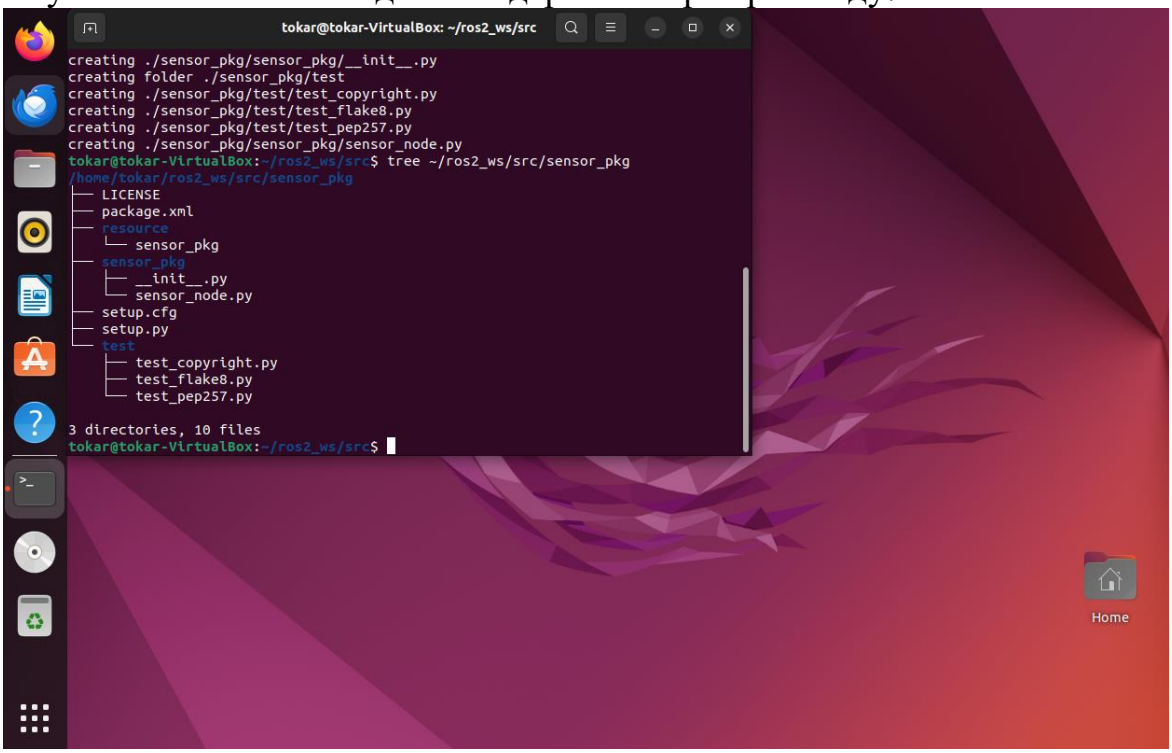


Рисунок 1- Створення пакету Sensor Node.

3. **Збірка workspace**
   Цей крок показує, як скомпілювати створений пакет і перевірити, що він готовий до роботи.

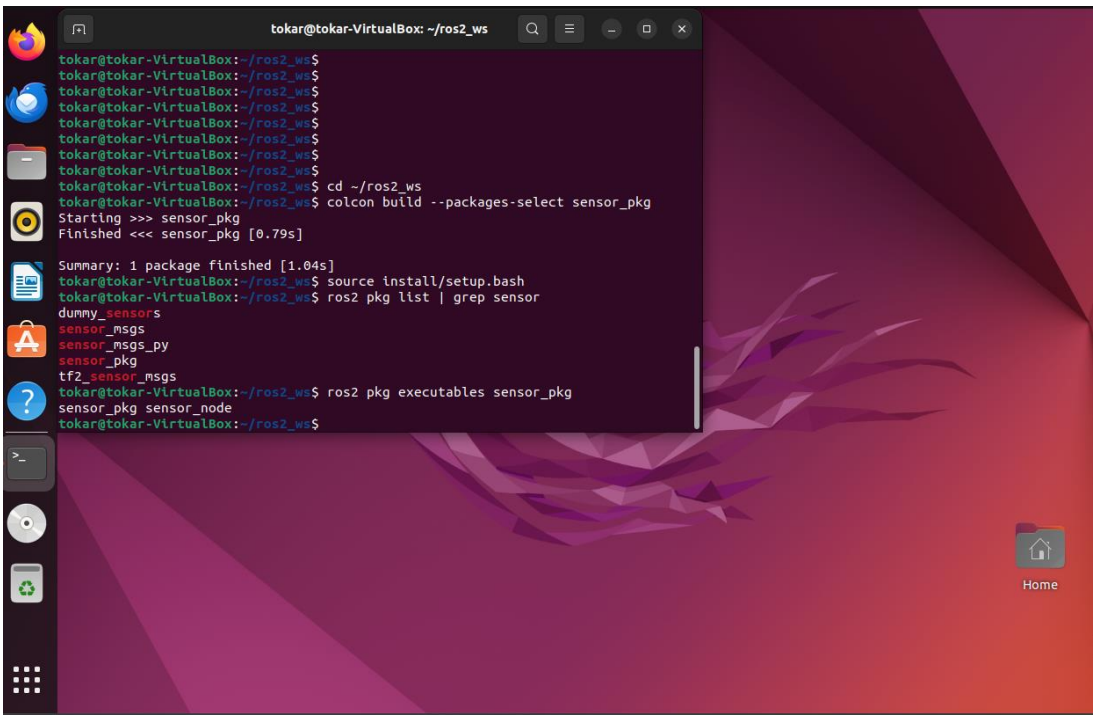


Рисунок 2- Збірка workspace

4. Відкриття проєкту у VS Code. Відкриваємо весь src, бо редагуватимемо два пакети (sensor_pkg і motion_control).

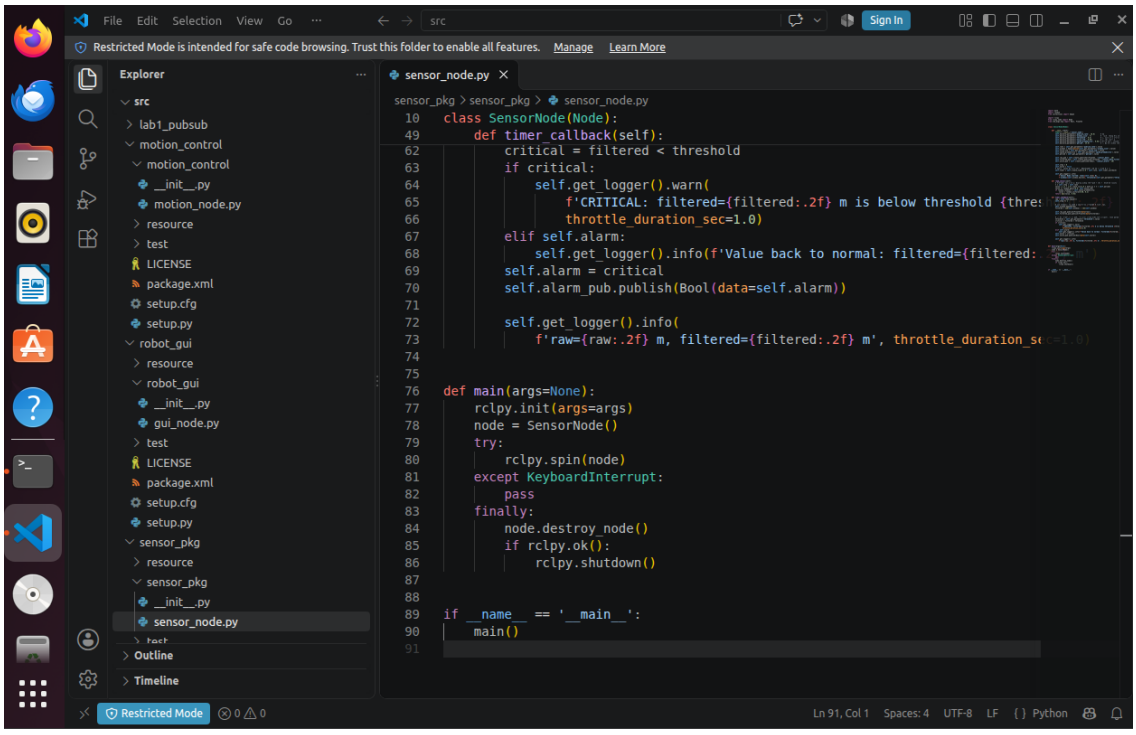


Рисунок 3 – Код сенсорної ноди

5. Перевірка package.xml та setup.py

Цей крок полягає у візуальній перевірці двох автоматично згенерованих конфігураційних файлів, щоб переконатися, що пакет налаштований правильно.

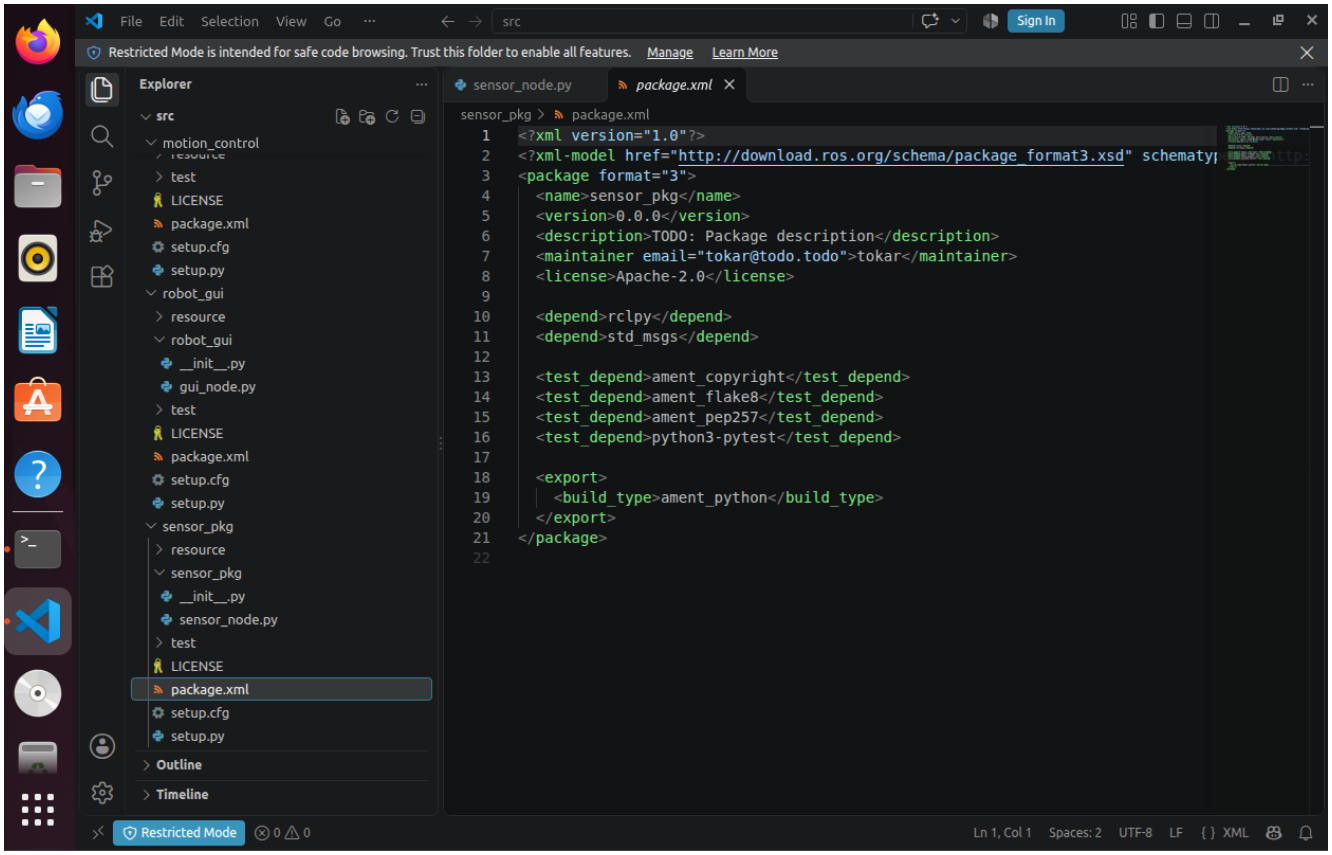


Рисунок 4 - Перевірка

6. Перевірка та запуск ноди сенсорної ноди

Під час запуску було перевірено що програма успішно працює. Він не просто генерує цифри, а й виконує логічні перевірки (порівнює з порогом) та коректно інформує користувача про стан системи за допомогою різних рівнів логів (INFO та WARN).

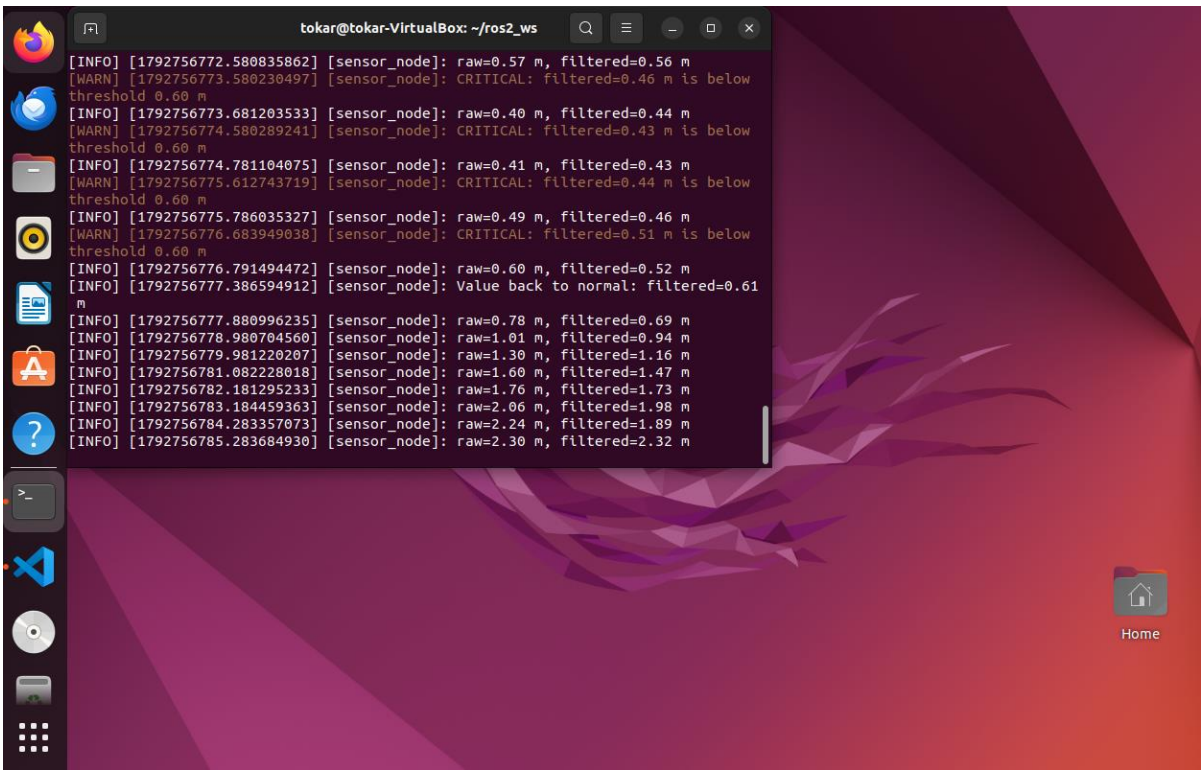


Рисунок 5 – Перевірка

7. Ноди, топіки і частота

Цей крок демонструє діагностику та перевірку архітектури ROS 2 за допомогою інструментів командного рядка з іншого терміналу, поки вузол продовжує працювати.

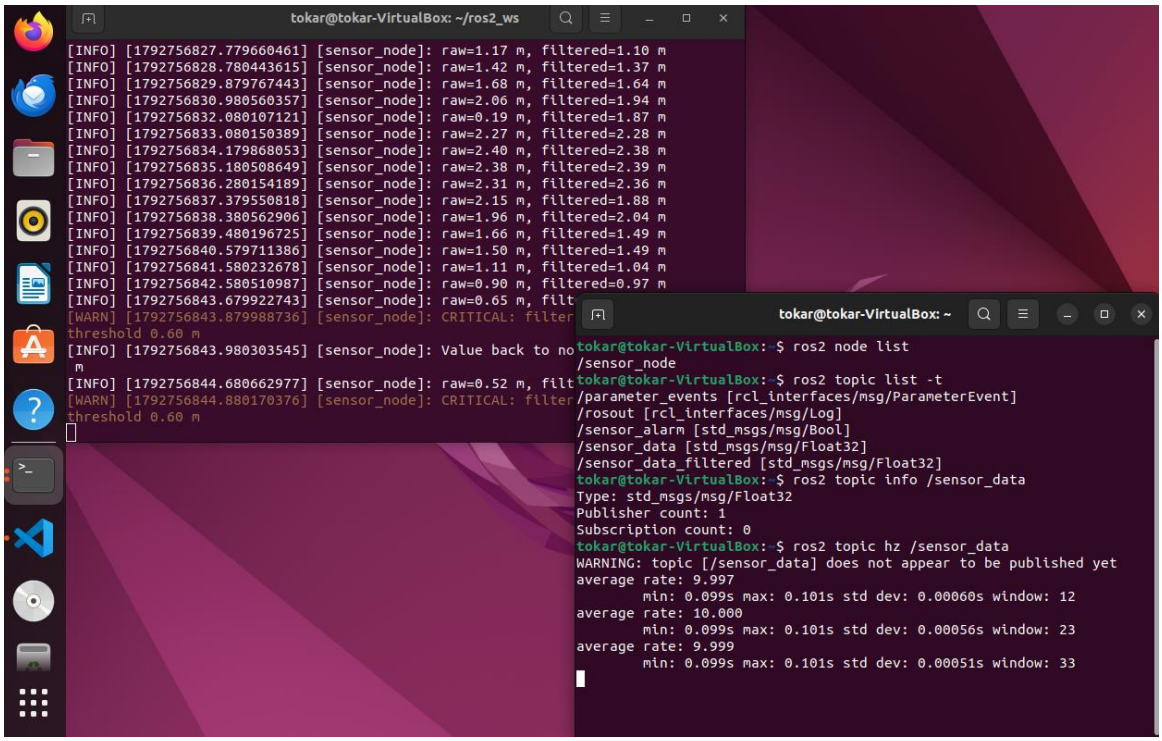


Рисунок 6 – Перевірка

8. Перегляд даних через ros2 topic echo

Це візуальний доказ того, що алгоритм фільтрації (наприклад, ковзне середнє), закладений у код, успішно працює і прибирає шум із показників сенсора.

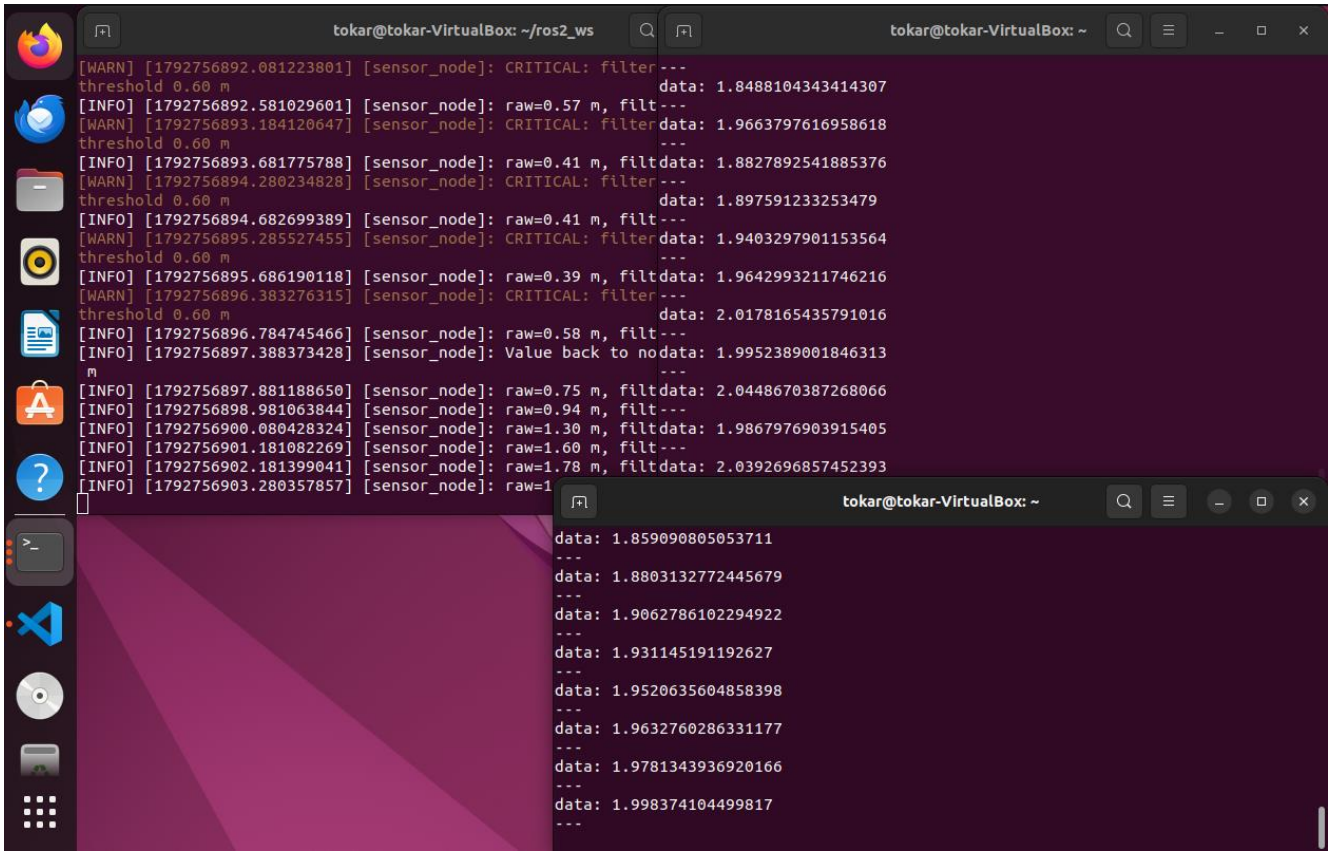


Рисунок 7 – Перегляд

9. Реакція системи на критичне значення (WARN)

Цей крок демонструє, як система реагує на небезпеку (критичне значення) і як можна динамічно змінювати налаштування вузла (параметри) без його перезапуску. Ввівши команду ros2 param set /sensor_node threshold 0.6, все повертається на круги своя. Показники сенсора знову стають вищими за поріг, і в лівому терміналі з'являється повідомлення [INFO] Value back to normal. Також він публікує логічні сигнали (True/False) в окремий топік для систем безпеки.

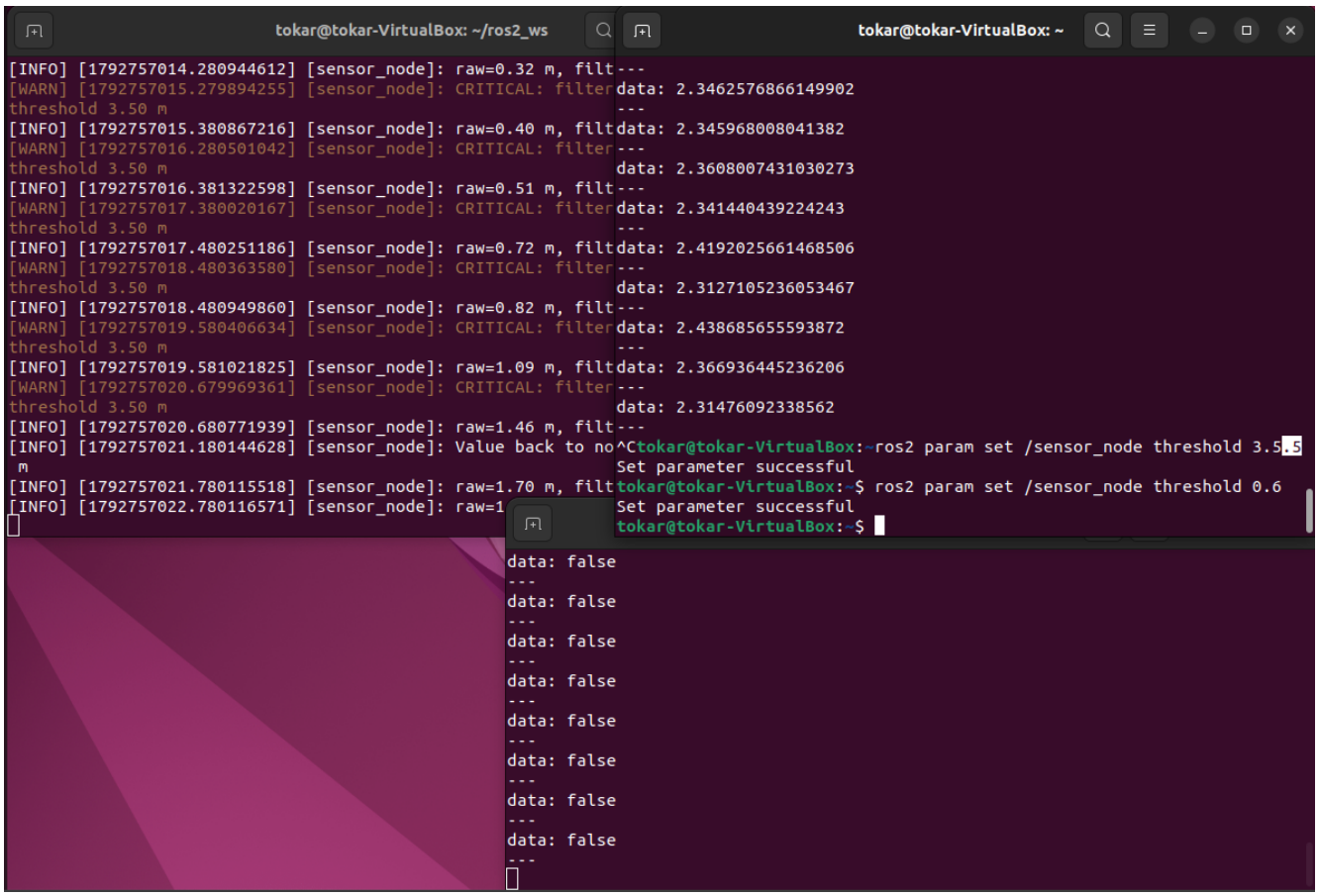


Рисунок 8 – Реакція системи

10.Оновлення motion_node.py

Цей крок описує об'єднання двох незалежних програм (вузлів) — сенсора та контролера руху — для створення розумної системи безпеки. Раніше ми створили сенсор, який вміє видавати сигнал тривоги (True/False) у топік. Тепер потрібно навчити інший вузол (який відповідає за рух робота) слухати цей сигнал. Якщо тривога активна (робот під'їхав надто близько до перешкоди), контролер руху має заблокувати рух уперед. При цьому рух назад і повороти залишаються дозволеними, щоб робот міг врятуватися від зіткнення.

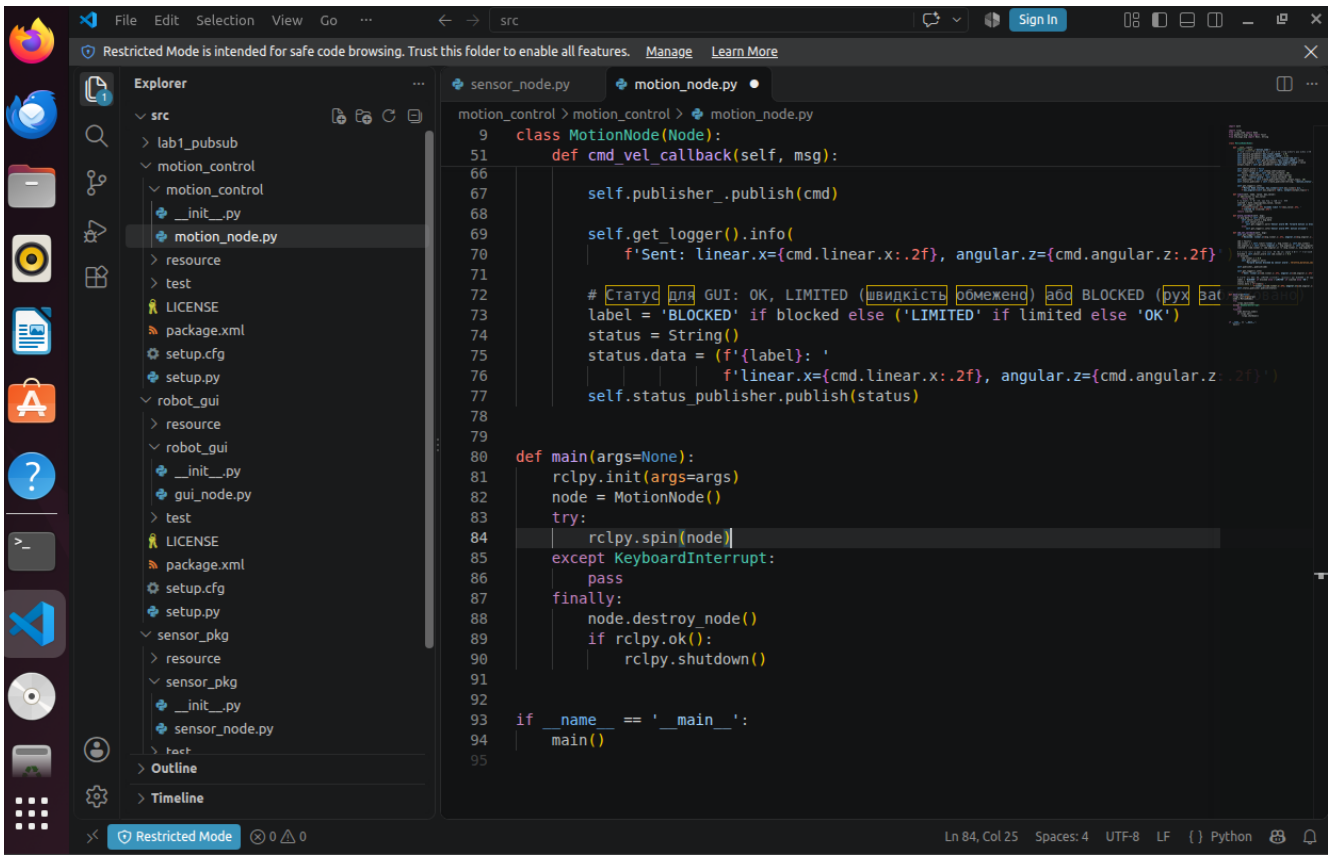


Рисунок 9 – Додавання змін у VS code

11. Перевірка змін

На цьому етапі відбувається фінальний тест інтеграції двох вузлів (сенсора та контролера руху). Перевіряється чи дійсно система безпеки працює: чи зупиняється робот (черепаха), коли сенсор фіксує небезпеку.

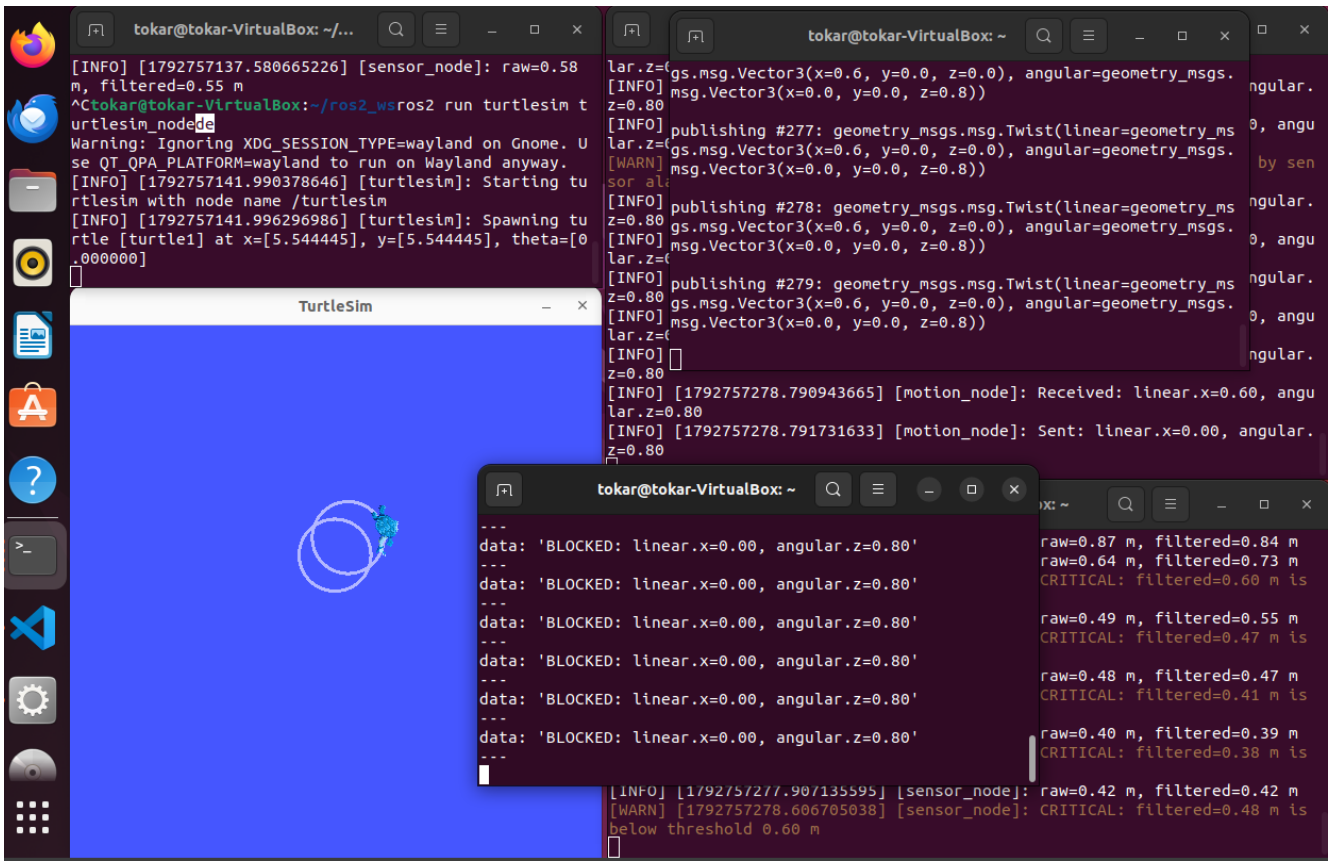


Рисунок 10 – Результат демонструє що система безпеки спрацювала ідеально: заблокувала небезпечний рух вперед, але залишила можливість маневрувати.

Висновок: У ході виконання лабораторної роботи було створено, налаштовано та зібрано пакет ROS 2 із вузлом сенсора, що генерує та фільтрує дані. Було проаналізовано архітектуру системи, перевірено роботу топіків і динамічну зміну параметрів, що дозволило імітувати аварійні ситуації без перезапуску програми. Ключовим етапом стала успішна інтеграція сенсора з вузлом керування рухом: розроблена логіка безпеки підтвердила свою працездатність у симуляторі, коректно блокуючи поступальний рух робота при виникненні тривоги, але залишаючи можливість повороту для маневру. Проведене тестування продемонструвало ефективність взаємодії незалежних вузлів через топіки та практичне застосування динамічних параметрів для побудови надійних систем робототехніки.